

COMP90025 Parallel and multicore computing

Project 1: Parallel byte-pair encoding (BPE)

Project 1 requires developing and submitting C/C++ OpenMP programs and a written report. Project 1 is worth 20% of the total assessment and must be done individually. The project is marked out of 20; the written report is worth 4 of those marks. The due date for the submission is nominally **11pm on 17 September 2026, the *Thursday* of Week 8**. Submissions received after this date will be deemed late. A penalty of 10% of marks per working day late (rounded up) will be applied unless prior approval is obtained. The weekend does not count as working days. Please let the head tutor know if there is anything unclear or of concern about these aspects.

1 Background

Byte-pair encoding (BPE) is the subword-tokenization algorithm used by most modern large language models. It learns a vocabulary from a corpus by repeatedly merging the most frequent adjacent pair of tokens into a single new token. Once trained, the vocabulary is used to tokenize new text. This year's projects combine to form a simple, parallel BPE trainer. In Project 1, you will write OpenMP (shared memory) code to perform the training pipeline:

1. Task 1 splits the input into *words*, counts how often each distinct word occurs, and then splits each word into its *characters* (a parallel prefix sum is useful here).
2. Task 2 repeatedly finds the most frequent eligible adjacent pair of tokens and merges it into one token, until no eligible pair remains (parallel sorting is useful here).

In Project 2, you will write MPI (message passing) code to extend this pipeline to a distributed setting with additional targets.

Since this is a simplified version of a real tokenizer, the requirements may differ from those of real-world tokenizers. This project is designed to be a good test of parallel programming skills with the simplified requirements.

2 The BPE algorithm, by example

We use the running example from the Hugging Face LLM course, section *Building a BPE tokenizer from scratch* (<https://huggingface.co/learn/llm-course/en/chapter6/5>). The same example is used in Tasks 1 and 2.

Task 1.1 counts word frequencies; the example corpus has five words with the following frequencies:

("hug", 10), ("pug", 5), ("pun", 12), ("bun", 4), ("hugs", 5)

Task 1.2 splits each word into its characters:

("h","u","g", 10), ("p","u","g", 5), ("p","u","n", 12),
("b","u","n", 4), ("h","u","g","s", 5)

Task 2 runs the merge loop. (R1)¹ A *pair* is two adjacent tokens inside one word. (R2.1) A pair is *eligible* only if it appears in **at least two distinct words**; (R2.2) a pair that occurs in a single word is never considered and never merged. (R3.1) The count of an eligible pair is its total number of occurrences across all words, (R3.2) weighted by word frequency. Each iteration merges the most frequent eligible pair everywhere, then recounts. The following steps demonstrate this task:

1. The most frequent eligible pair is (u,g) with count **20** (it occurs in **hug**, **pug** and **hugs**; see (R3.1) and (R3.2)). Merging everywhere turns every (u,g) into the single token **ug**; the vocabulary now includes **ug**.
2. The most frequent eligible pair is now (u,n) with count **16** (in **pun** and **bun**). Merge to **un**.
3. The most frequent eligible pair is now (h,ug) with count **15** (in **hug** and **hugs**). Merge to **hug**.
4. Every remaining adjacent pair, (p,ug), (p,un), (b,un), (hug,s), occurs in exactly *one* word, so no eligible pair remains and training stops.

(R4.1) The final tokens that appear, with their counts ((R4.2) descending), are

(p, 17), (un, 16), (hug, 15), (s, 5), (ug, 5), (b, 4).

Corner Cases. Whenever two things have equal counts, compare their strings with `strcmp`: if `strcmp(s1,s2) < 0`, put **s1** first ((R5.1) merge **s1**'s pair first, or (R5.2) list **s1** first in the output); otherwise put **s2** first. In the example **s** and **ug** both occur 5 times; since `strcmp("s","ug") < 0`, **s** is listed before **ug**. (R5.3) The input is case-sensitive. (R5.4) If two occurrences of the pair overlap, merge them greedily from left to right (a word (a,a,a) under the pair (a,a) becomes (aa,a), not (a,aa)); recount from scratch in the next iteration.

3 Data Specification

1. The input is a single large ASCII text file of arbitrary length, containing any bytes except NUL (0x00).
2. A *word* is a maximal run of characters that are not ASCII whitespace, where ASCII whitespace is exactly one of: space (0x20), tab (0x09), line feed (0x0A), carriage return (0x0D).
3. **Whitespace is a separator to discard.** Whitespace bytes are stripped: the words are concatenated with no whitespace in between. *This is a naive simplification of real-world tokenizers, so that you are not distracted by whitespace handling and can focus on the parallel implementation.*
4. Word splitting is deterministic and independent of the number of threads. This means that no matter how many threads are used, the same words are produced.

4 Assessment Tasks

(R6.1) The skeleton code provides the data structure that holds the results of each task, and you must fill it in. This is mandatory, because each task is marked on its output.

(R6.2) You must implement all parallel algorithms yourself; do not use libraries that come with parallel algorithms.

(R6.3) You may use your own data structures for any intermediate results, but the final output must be in the provided data structure.

¹The circled numbers indicate the rules that you have to follow.

1. **Task 1: word counting and character splitting (6 marks: 2 correctness, 4 speed).** The task has two parts, Task 1.1 and Task 1.2.

- (a) **Task 1.1: word frequency counting.** Read the input file, split it into words (stripping whitespace as in the Data Specification), and count how many times each distinct word occurs.
- (b) **Task 1.2: character splitting.** Split each distinct word into its characters, each character being a single byte, and keep the characters in the word’s original order: `h u g 10`, not `h g u 10`.

2. **Task 2: the BPE merge loop (8 marks: 2 correctness, 6 speed).** Starting from the word-character splits of Task 1.2, run the iterative BPE merge loop:

- (a) Count every adjacent pair (a, b) of tokens inside every word; a pair’s count is the total number of occurrences across all words, weighted by word frequency;
- (b) A pair is *eligible* only if it appears in **at least two distinct words** (a pair that occurs in a single word is not considered);
- (c) If no eligible pair exists, stop;
- (d) Otherwise select the eligible pair with the highest count; ties are broken by comparing the merged strings (the concatenation of the pair’s two tokens) with `strcmp` as in the tie-breaking rule above (the lexicographically smallest merged string is merged first);

Write `output.txt`: one line per token that appears in the final tokenized corpus, of the form

```
token count\n
```

The lines are ordered by decreasing count; when two tokens have equal count, the one that is lexicographically smaller (smaller `strcmp` value) comes first. For the example corpus, `output.txt` is byte-exact:

```
p 17
un 16
hug 15
s 5
ug 5
b 4
```

The rule is fully deterministic: for a fixed input file there is a **unique correct answer**. Your Task 2 output must match it byte-exactly.

3. **Task 3: SLURM benchmarking (2 marks).** Fill in a SLURM script named *benchmark.slurm* that benchmarks your implementation on Spartan. The script must generate a file called *results.csv* with the following headers: filename, file_md5, version, partition, threads, walltime, ram, output_md5. The script itself is part of your source submission, but the *results.csv* it generates is produced by running it on Spartan and is not part of your submission.

A missing SLURM file, or a SLURM file that does not generate the correct results, earns you zero marks for this task.

4. **Written report (4 marks).** Use LaTeX to write a report of **at most 2000 words** (not including figures, tables, diagrams, pseudocode or references) with the following sections and (where given) lengths:

- (a) Introduction (100–200 words): a few sentences on what was done in the tasks and the headline result (e.g. the speedup achieved).

- (b) Results: fill in the results-table template with your own measured results, and include the required scaling figure. A short paragraph stating the measurement protocol (machine, inputs, thread counts, repetitions) is required so the results are interpretable. The text should be only enough to explain what is presented, not to analyse the results.
- (c) Analysis and Conclusion: critically analyse the results; discuss what they do and do not show, where the parallel speedups come from and where they break down, and what can be concluded from them. If you have a good reason to write more or less than the guidelines, you may.

A LaTeX project template will be provided: upload it to Overleaf and fill in the required sections or rewrite the whole sections as needed.

5. Other requirements.

- (a) **(R7.1) Correctness is the gate for Tasks 1 and 2: an incorrect output scores 0 speed marks.**
- (b) **(R7.2) Parallelism is another gate for Tasks 1 and 2: a sequential implementation earns 0 correctness and 0 speed marks.**
- (c) **(R7.3) Version control is required: the skeleton is a git repository, and you must commit your work to it reasonably frequently. The author email must be your student email address. Up to 50% of your final marks may be deducted for failing this requirement.**
- (d) **(R7.4) Compilable code and no runtime errors: your code must compile and run without errors on Spartan. Up to 50% of your final marks may be deducted for failing this requirement.**

5 Mark Allocation

The software component is worth 80% of the project marks and the written report 20%. The project is marked out of 20:

Component	Marks	Weight
Task 1 (word counting + splitting): 2 correctness + 4 speed	6	30%
Task 2 (BPE merging): 2 correctness + 6 speed	8	40%
Task 3 (SLURM benchmarking)	2	10%
Written report	4	20%
Total	20	100%

Speed-up marks in total are 10 of the 20 marks (50% of the project). Speed-up marks are awarded on a curve relative to the teaching-team sequential baseline reference implementation on the fixed reference machine (Spartan). The expected speed-up grades are:

Grade	Speed-up relative to the baseline
H1	Speed-up likely to be $O(p/\log p)$ for p processors for sufficiently large problems; probably faster with two processors than the best sequential algorithm.
H2	Speed-up likely to be $O(\sqrt{n})$ for sufficiently large problems.
H3	Some speed-up for large problems and large numbers of processors.
P	Performance is slightly better than the provided sequential algorithm on big inputs.

6 Submission

Submit exactly **two files** on Canvas (the submission option will be made available closer to the deadline). Replace `studentid` below with your student ID (digits).

1. `studentid.tar.gz`: a single tarball of the **git repository** you worked in locally, based on the provided skeleton repository. When extracted, it must reproduce your working folder, including its git history, so that the version-control requirement ((R7.3)) can be verified. It contains **only source code**:
 - the complete skeleton with your parallel implementation of the parallel API entry filled in, and your name, student number, and examples of how to test the correctness and performance of your program documented in **README.md**;
 - a **Makefile** based on the skeleton. Do not alter the provided targets; you may add your own.
 - `benchmark.slurm` (Task 3).

(R8.1) **Do not include any built files, input files or output files** in the tarball: no compiled binaries or object files, no sample datasets or test-case inputs, and no generated outputs such as `output.txt` or `results.csv`. The teaching team compiles and runs your submitted source code on Spartan, with their own inputs, to verify correctness and speed. Keep generated artifacts out of the repository (e.g. via a `.gitignore`), so that the committed history contains only source code; the only measured results you hand in are those reported in your written report.

(R8.2) In the Acknowledgements section of **README.md**, describe any assistance you received (from peers, tutors, lecturers, friends, etc.). You must acknowledge any help from AI (e.g., GitHub Copilot, Claude Code, Codex) and list how it was used, including the detailed prompts and responses. If you used a web-based AI tool (e.g., ChatGPT), share the conversation and include its link in the Acknowledgements. If you did not use any AI, state that explicitly.

2. `studentid.pdf`: your written report, compiled with the provided Overleaf LaTeX template.

(R8.3) These two files are the entire submission; do not upload anything else unless asked by the teaching team. (R8.4) The report is submitted as the separate PDF, not inside the tarball.

7 Other Information

The Skeleton code contains a `README.md` file; due to the limitation of PDF rendering, the Skeleton code is considered as the extended version of this document with detailed requirements. You should read the latest version of `README.md` file carefully and follow the instructions in it.

Any requirements updated by the teaching team will be announced on Canvas/Ed/updated specification file. Those updates will be considered as a part of this specification. You are responsible for checking for updates and following them.